

3DOF Proportional Navigation Intercept Algorithm With Kalman Filtering

Features:

Randomized Target Path

- By discretizing the control volume into 4 regions and generating a random point for each region, a spline is created through the random points, creating a randomized path each time the code is run.

Proportional Navigation Scheme

- Acceleration perpendicular to the missile velocity in the direction that cancels the LOS angular rate.
- Uses closing velocity as a gain for commanded acceleration
- Uses PN gain, N , set to 5 for aggressive maneuvers.

Boost Phase and Glide Phase

- During boost, the missile accelerates up to glide speed. During boost, the missile does PN, but also accelerates in the forward direction.
- Once the missile reaches max speed, which is set to 5000 m/s, it switches to glide phase, where the missile only does PN and does not speed up anymore. Total speed increases slightly due to perpendicular acceleration from the PN algorithm.

Acceleration Lag

- In real systems, acceleration cannot be instantaneous. Control surfaces lag, and physics lag control.
- Introduced a first-order lag for acceleration, so an instantaneous commanded acceleration given by the PN algorithm cannot be instantly implemented.
- This means the true acceleration lags slightly behind the commanded acceleration.

Acceleration Clamp

- Real missiles typically operate with a maximum acceleration of 40-60g, so the model ensures accelerations do not exceed the user specified maximum.

Kalman Filter Algorithm

- Input: Noisy relative position data (simulating sensor noise from radar or seeker).
- The state is defined as 3D position, velocity, and acceleration, and is propagated forward using a motion model.
- Jerk is represented implicitly as white (Gaussian) process noise acting on acceleration.
- Sensor data from relative position corrupted with white (Gaussian) noise.

- The Kalman gain, computed from the state covariance (P) and measurement noise (R), determines the optimal weighting between the prediction and the measurement based on their estimated uncertainties.
- Output: estimated position, estimated velocity, and estimated acceleration, filtered from the noisy position data.
- The X, Y, and Z positions are plotted just before impact, showing the input positional data versus the data after it has passed through the Kalman filter.

Live Intercept Plot

- Uses MATLAB's "drawnow" command to plot the data as if it were updating live.
- Shows the position of the target and the missile.
- Utilizes a "skip" variable that allows the plotter to only plot every "x" point, where "x" is the skip value. Useful for optimization and decreasing plot time.
- Stops plotting at the intercept, and indicates if/when the missile intercepted the target. Also plots an asterisk, representing fuse detonation.
- Records missile velocity each iteration so boost and glide can be visualized.

G - Pure proportional navigation (PPN) with boost and glide phase

N - 9-state Kalman filter for relative position

C - 1st order lag and acceleration clamp

Shortcomings

No Physics

- This algorithm only models position, velocity, and acceleration. The bodies themselves do not experience any forces. Thus, this simulation mostly represents the navigation aspect of GNC.

Only 3DOF

- Both the target and missile are treated as points that simply translate in 3D space. Rotation schemes, such as Euler rotation or Quaternions, are not utilized here because frames are not tracked.

Missile Position is Treated as Known

- The missile's position data is not corrupted, as if it always knows exactly where it is. Only the target data is corrupted. In reality, the missile would have its own set of sensors to determine its position, which are also subject to noise.
- The algorithm would need another Kalman filter for the missile's own position, velocity, and acceleration.
- Compounding error such as IMU drift is not modeled.

No Simulated Control Systems

- The missile modeled here does not have any mass properties or control systems. Simulating the controls of the missile adds multiple layers of complexity that are just too much to tackle in 1 week's worth of free time.

Elementary Control Scheme and Filtering

- PPN is about as foundational as it gets for missile guidance.
- Many more robust and sophisticated navigation and filtering algorithms have been invented, and it is the job of GNC teams around the globe to keep inventing better algorithms.

Future Development and Next Steps

Immediate Next Steps:

- Missile's own navigation. This code assumes a perfect missile position. Would want to corrupt this data and add some error runoff to simulate sensor noise and drift. The missile needs its own Kalman Filter.
- Monte-Carlo and success analysis for tuning. I want to wrap this algorithm in loops to vary parameters and model success trends.

Improvements further down the line:

- Aerodynamics Modeling - Modeling the aerodynamic forces themselves as they interact with the missile.
- Inertial Modeling - Modeling how the missile reacts to forces on its body.
- 3DOF-->6DOF by calculating reference frames and using quaternions.
- Switch from Pure PN to Augmented PN.

To improve this algorithm, I would want to address all of the shortcomings mentioned above. But, doing so would take me several more weeks, if not months. The reason GNC teams are so important in aerospace is that it is not an easy problem to solve; coding this project really showed me that. This algorithm is a great tool for understanding the basics of missile guidance, but it is not useful for much in the real world. I would use MATLAB Simulink to create an inertial model of the missile, which would allow me to solve for the aerodynamic forces in flight. My background is in aerodynamic theory, so this aspect really interests me for future work. Using an algorithm like the one I developed here would be useful to determine the path that the missile needs to take for intercept. Now, I just need to work on how a real missile should follow that path.

Daily Work Log

I am starting this project with the intention of learning more about GNC algorithms. Although I have a basic understanding of controls, I have never developed an intercept algorithm, and I'd like to bridge that gap in my knowledge so that I can do it professionally. I have 6 days between my NG GNC interviews to learn and apply as much as possible.

Day 1

I spent some time trying to scope the project. I want the model to be as robust as possible, but I also have to be realistic. I have never done this before, so I need to learn how. I will also be balancing end-of-semester work along with my job. As much as I want to do it, a 6DOF Simulink model with a controls simulation just feels like more than I can handle in 6 days.

I think the 6DOF Simulink model that I initially had in mind would be better reserved for a multi-month-long project where I can explore the details of GNC. Right now, I want to focus on the basics.

I think a good goal for the next 5 days is to create a MATLAB script that simulates an HTK missile intercepting a target that follows a random path at hypersonic speed. I will model simple actuator lag and aero forces so I can implement a controller and add noise to simulate imperfect data from the sensors.

I want to learn and/or apply:

- Some sort of algorithm that simulates noisy position data
- Kalman filter
- Either LQR or PID controller
 - More familiar with PID, need to study LQR
- Proportional Navigation to predict intercept trajectory

Progress:

On day 1, I only got to work on the project for about 2 hours. During this time, I learned as much as I could about writing algorithms in MATLAB by watching YouTube videos and reading online articles. I developed my basic understanding of Kalman filters and proportional navigation.

For MATLAB, I created a script that simulates a hypersonic target that moves along a semi-randomized 3d spline in a 300x300x300 Km 3D space. It moves generally from 300km to 0km in the y direction, while having more freedom of movement in the x and z directions. I "animated" the target's movement by using the "drawnow" command.

Tomorrow I will try to focus on developing the Proportional Navigation algorithm, without any noise, aero forces, or acceleration lag. I just want to see that I can make a predictive intercept algorithm, and I'll add the complexity later.

Day 2

Not much time to work today, less than an hour, but still going to try to make some progress learning and developing.

Proportional Navigation

First, just to get some sort of working guidance algorithm, I will make a PN loop with a point that accelerates toward the target by minimizing line of sight rotation. **LOS is the vector that connects the missile and its target.** PN works by accelerating the pursuing missile at a rate that keeps the LOS angle constant (makes the rotation rate zero) until eventual intercept. See ref [1]. This works because if LOS has no rotation, the target is aligned with the missile reference frame, and only gets closer as the missile travels. This means the missile must be on an intercept path.

N is the navigation constant. This determines how aggressively the missile responds to any LOS rotation. Higher N requires more aggressive movements, so it typically uses more energy and requires a more robust missile. $N = 4$ or 5 is good for maneuvering targets because strong correction is needed. $N=2$ and $N=3$ are good for efficiency. Pretty similar to a gain in a typical controller. This is just to adjust the acceleration, though.

If I am going to develop a 6DOF control scheme, I could use unit quaternions to align the missile frame with the LOS frame. Quaternions are useful because there are no rotation singularities, unlike Euler rotation (as learned in Robotics). This would give me missile orientation relative to the global coordinate system (or target coordinate system). It doesn't make too much sense to use them yet since I don't have a physical missile body. Right now, I only care about missile position. The simulation is too simple to consider equations of motion and seeker head pointing.

Since I am not considering missile orientation yet, I will just find the LOS vector and accelerate my point in the direction of the vector. This will all be done in global coordinates. This is as simple as it gets for GNC. There are no physics yet, just a point chasing another point.

I actually got it working today. Worked for way longer than I should have. I added the missile portion and the PN part of the code today.

Set missile initial position and velocity. Set kill range, set max speed. Set up a boost phase and a PN phase. Boost gets the missile to max speed, and PN takes over after. PN accelerates the missile normal to LOS using $\dot{\phi}$, which is the LOS angular rotation rate. Plotted the missile trajectory along with the target trajectory in the drawnow plot to see a live intercept in 3d. It displays the minimum distance, and if the intercept is successful.

Had a bug I couldn't figure out for a while where it looked like the missile was being repelled from the target when it got close. Figured it out. I was trying to accelerate along the line of LOS during the PN phase, but this was just making the missile go the wrong direction because it already had a lot of velocity, and was not properly accelerating to minimize LOS rotation. I actually had to take the cross product of LOS rotation rate ($\dot{\phi}$ I called it) and the LOS unit vector so the rocket could be accelerated perpendicular to LOS. As it boosts and gets closer to

the target, it develops a lot of velocity in the target's direction. I had to apply the acceleration perpendicular to the rotation rate instead. This is TRUE proportional navigation (**TPN**). Pure PN is when acceleration is perpendicular to the missile's velocity vector, not the LOS. TPN works better here because I have direct access to the LOS, but PPN is probably better for real missiles where aero control surfaces push with respect to the missile's velocity vector. Acceleration is about perpendicular to motion in both schemes to maintain constant speed.

Takeaways: LOS vector, LOS rotation rate, acceleration orthogonal to LOS in the direction of rotation rate, control speed as the time varying gain, N as the gain parameter

- LOS vector
- LOS rotation rate
- Acceleration orthogonal to LOS in the direction of LOS rotation for TPN
 - Orthogonal to the missile velocity vector for PPN
- Control speed - acts as a time-varying gain. If the difference in speed between the missile and target is larger, accelerate faster.
- N is a gain parameter that sets how aggressive the PN works. Most intercept algorithms use 4 or 5 because they need to be aggressive, but not too aggressive. Above 5 uses too much energy and can accelerate beyond physical limits, also, diminishing returns. 2 and 3 are good when efficiency is more important than aggressiveness/accuracy.
- Boost phase and PN/glide phase

Day 3

Read some additional resources and found that, in general, PPN is a better guidance law than TPN. I thought that TPN would be easier to implement and be more effective, since I was not working with control surfaces or physics, but I implemented a PPN algorithm today, and sure enough, it works just as well, if not better than the TPN algorithm I originally had. The reasoning for using missile velocity as a gain for acceleration is elegant and logical. If the missile is moving faster, I want to accelerate faster to make sure I do not miss. Also, PPN would be much easier to implement, because the missile can easily accelerate perpendicular to its own velocity vector in a real system. For those reasons:

TPN > PPN.

The real goal for today is to introduce a Kalman filter. I want to simulate noise in the system and practice removing it. I should be able to add some white noise to the position vector and use a Kalman filter to not only calculate velocity, but also remove the noise. I'll plot the noisy vector along the filtered vector to visually identify the result.

“The Kalman Filter is a state estimation algorithm that provides both an estimate of the current state and a prediction of the future state, along with a measure of their uncertainty. Moreover, it

is an optimal algorithm that minimizes state estimation uncertainty. That is why the Kalman Filter has become such a widely used and trusted algorithm.” Kalman Ref [3].

Just didn’t get around to filtering today, spent most of my time learning.

I also had to troubleshoot a bug where sometimes the missile was overshooting the target and coming back around. This was happening because I was checking for the minimum distance between the target and the missile, when what I really wanted was the first point that was within intercept range. What would happen is that the missile would get well within intercept range, but keep running and find a minimum that was slightly lower, thus causing the algorithm to optimize towards the intercept that happened later, instead of the first one.

Day 4

Added a time delay to the acceleration called “tau” to simulate the delay from control systems. This is the solution to the general lag transfer function modeled by $(1/(\tau*s + 1))$, which in the time domain looks like $\tau*\text{jerk} + \text{acceleration} = \text{command acceleration}$. The actual acceleration is carried as a state variable in the loop, so that instantaneous acceleration isn’t possible. The acceleration updates from the time-delay weighted acceleration of the previous iteration. Ran into an issue where the acceleration computed using the forward Euler solution in the time domain gives inaccuracy for some time step ratios. Used zero-order hold discretization, which scales the solution by an exponential, which I called alpha. Had to learn about this, and to be honest, still don’t understand it super well. I got it working, but if I work in GNC, I want to better understand state-space variable filtering.

As a refresher, a state is the minimum set of variables that completely describes a system. For example, a moving object at constant speed only needs position and velocity to be defined at any given moment.

Kalman filtering

Finds an optimal estimate of a state vector by balancing the prediction of a dynamic model with measurements from real sensors. The real sensors typically introduce significant noise to the data, and the Kalman filter can help smooth this data out by developing the relevant model. If the sensor data is very noisy, trust the model’s data more, and if the sensor data is stable, trust the sensors. The filter determines how noisy the sensor data is by using covariance (statistical uncertainty). A default Kalman filter works best on noise that is normally distributed, which is called Gaussian noise (essentially random white noise). The Kalman filter is useful because it can predict states that cannot be physically measured and can smooth noisy data.

In Kalman filtering, there are two main sources of uncertainty:

1. Process noise. Uncertainty in the model. (Q matrix)
2. Measurement noise. Uncertainty in the sensors. (R matrix)

Both forms of uncertainty must be considered, so two covariance matrices are created, Q and R, to represent how much noise is in the system. If the covariance is large in Q, weight the data from sensors more. If R has a large covariance, weight the predictive model data more. This algorithm ensures that the data set with the least covariance is favored at all times, which is what smooths the data towards the “true” data.

The main problem with a Kalman filter is that it can not adequately predict targets that maneuver quickly. This is because it relies on a state vector and state transfer matrix, but the state is never truly defined. For example, if the missile is highly agile, it cannot be defined by just position and velocity, because there is acceleration. It can't be modeled by position, velocity, and acceleration, because there is jerk. And so on. The Kalman filter *can* tell you exactly where the missile is and how fast it is going at any given moment, but the target can introduce external inputs into the system, making the problem truly unpredictable for the future.

As discussed, the Kalman filter creates a model that evolves according to the predictive model, plus random noise. Well, for a highly erratic target, the predictive model cannot possibly be very accurate, since the erratic nature of the target is likely due to erratic control inputs, which have nothing to do with the state of the target. This is why the question is harder than it seems. Some algorithms can run multiple Kalman filters in parallel to attempt to predict the target's behavior. Others set the Q high, meaning the model has high uncertainty, so the data is trusted more heavily than the model.

This is turning out to be a more interesting problem than I initially thought.

I will introduce a Kalman filter by corrupting the position data with noise, and then building an algorithm to predict velocity and smooth the data.

Still not done with this at the end of day 4. I just don't fully understand the algorithm. I know what it does, what it's for, and why missiles use it, but I am having a very hard time actually implementing the algorithm into my code.

Today I added the time delay factor, fixed bugs, and rearranged the syntax to look a bit nicer. Nothing crazy, but still an improvement. Hopefully, the Kalman filter will be done tomorrow, and I will consider that a good stopping point.

Day 5

Kalman filter model

Assumes that the target does not experience instantaneous, hard acceleration. So, the acceleration is modeled as uncorrelated noise, since I can't predict the future acceleration based on previous time steps. This should have the real position data weighted more in the filter. The Q matrix is the covariance of the noise, which comes from the equations of motion. The effect of an unknown acceleration over the interval dt can be written as $\Delta(x) = (\frac{1}{2})a*dt^2$, and for velocity, $\Delta(\dot{x}) = a*dt$. This is the noise input vector, G. $Q = GG^T*q$, where q is the noise gain.

$$\mathbf{Q} = q \begin{bmatrix} \frac{dt^4}{4} & \frac{dt^3}{2} \\ \frac{dt^3}{2} & dt^2 \end{bmatrix}$$

This gives a matrix that looks like this for 1D motion. This is covariance due to unpredictable acceleration over one time step. I just propagated the model into 3D to make a 6x6 matrix. This is the covariance model I'll use to model unknown acceleration as noise in the model.

R - The sensor noise covariance matrix doesn't require any derivation, and since I am just using position is simply the standard deviation of sensor noise in each axis, along the diagonal of the matrix. So, Q is 6x6, where R is only 3x3.

Q answers: how wrong can my model be at each timestep due to unknown inputs?

R answers: how wrong can my sensor be at each measurement?

Since both the target position and the missile's own position will have sensor noise, I introduced sensor noise to both and then added them to the R matrix. This is valid because the noise is random, normally distributed, and completely independent. It's sort of like, instead of the missile being unsure of where it is, it is just more uncertain of where the target is. This is random on every loop and does not have any memory, so cumulative errors, such as IMU drift, are not accounted for here!

In a better simulation, I would have at least two Kalman filters. One to estimate the missile's position, and another to estimate the target position. However, since this is a very low-fidelity model, I am just going to add the sensor errors for simplicity. Should give me about the same effect I am looking for. Not adding IMU drift for the same reason; it's just overkill for a simulation that doesn't even consider forces.

H - The measurement matrix, H will be 3x6, since I will be representing a 6x1 vector in 3d space. The 6 values in the state vector are position (3) and velocity (3). The velocity half of the measurement matrix will be zero because no velocity is being observed.

F - The state transition matrix is just the matrix that relates the new position to the old position, updating with the velocity at each step. New position = old position + old velocity*dt.

P - Preallocation for state error covariance matrix. Higher uncertainty for velocity since it isn't measured. P represents the confidence in the current state estimate (position and velocity in this case). It will be updated in each step inside the for loop to balance trust in the model vs trust in

the sensors. Using covariance tracks how errors propagate over time, which allows the filter to allocate weight for each model.

Got the filter working, but it needs some serious tweaking. Right now, the filter is causing the missile to miss more often than before because, instead of using true position and velocity, I am now using a noisy position reading and predicting velocity. So, this behavior is expected. I made my data worse, and the algorithm performs worse. However, it seems like the filter data is significantly lagging the turns of the target along the randomized spline. This is making the filtered data always overshoot or undershoot the true position, since the predictive model I derived assumes constant velocity, which obviously isn't the case. Need to add acceleration into my model so the filter can predict curved pathing.

Made a whole bunch of small tweaks to the model to get it more accurate and it is in a good place now. Filter now uses jerk as uncertainty instead of acceleration, and works much better. I also made the initial velocity vector in the direction of the first LOS, so it would start more accurately. For the kalman filter, the missile's true position is assumed to be known, while the target's measurements have noise. In reality, both would have noise, and both would have a filter, but this feels too complex. The intercept validation is the true relative distance of the missile and target. This is because the fuse should operate on a proximity sensor, not on an estimate from the Kalman filter. This means that whether the intercept is successful is dependent on how close the missile actually gets to the target, not how close it "thinks" it is.

Added a plot to visualize the Kalman filter, and had it zoom in on the moment before intercept, which should matter the most. Formatted so it shows just before intercept, and the noise is visible.

Bugs and Fixes

1. Spline produces wildly unrealistic behavior. Swirling, instantly cutting back, leaving test domain, etc.
 - a. Restricted randomized points to 4 distinct bounds, ensuring the rocket generally follows a path from +x to -x.
2. The missile seems to be getting “repelled” away from the target
 - a. I accidentally accelerated the missile in the wrong direction. It needed to be accelerated along the direction the LOS is moving, perpendicular to the LOS or the missile’s velocity.
 - i. Made this change, and the algorithm worked for the first time
3. Plotting extremely slow
 - a. Added a “skip” condition that checks for the remainder by division. Effectively plots every “x” point where “x” is the skip variable. Speeds up plotting time significantly.
4. Unbounded acceleration
 - a. Went away when I set a max speed and separated the phases into “boost” and PN
5. The missile was going past the target before coming back around and finally intercepting
 - a. The algorithm was checking for the minimum distance between the target and missile, and claiming that to be the intercept. The problem was that when the missile got within kill range of the target, but got closer at a later iteration of the loop, it would ignore the first intercept and plot the global minimum. Since I only care about the first time the missile is within intercept range, I added a for loop that breaks when the first intercept range is found. If no intercept is found, the code also has an if statement to let me know.
6. $TPN > PPN$
 - a. Not really a bug, but I read that for the most part, PPN is preferred over TPN, so I adapted the code to use PPN instead. Acceleration in the directed perpendicular to missile velocity, instead of perpendicular to LOS.
7. Missile missing about 10% of the time
 - a. Time step, dt , was too large. The missile was sometimes covering the entire intercept radius in one time step, causing it to “miss” because the position between time steps was not stored. Made the time step smaller and the issue was resolved.
8. The algorithm lags behind the target movement after introducing the Kalman filter.
 - a. Added acceleration to the state model and assumed jerk as the noise. Since the target follows a relatively smooth spline, adding acceleration to the model fixed the old filter’s inability to predict turning.
 - i. Massively improved success rate
9. Missile start behavior is extremely erratic in about 1/20 runs.

- a. Since closing velocity, which is used as a gain for acceleration, is computed using the Kalman filter estimates of LOS and relative velocity, the closing velocity was calculated to be wildly wrong before the Kalman filter settled.
 - b. Used missile velocity as the gain for the boost phase. Works relatively the same, since the missile is speeding up as it gets closer to the target anyway. Switches back to closing_vel during glide when the Kalman filter is fully settled.
10. Missing about 1/50 runs.
- a. Increased process noise/uncertainty, but had to make it unreasonably high
 - b. Fixed kalman filter initialization to better model the uncertainty. Allowed me to lower q value to something more reasonable.
 - c. Increased the amount of time steps. I think what was really happening was that the time step was just too large for the speed. Adding more time steps lowered chance of missile slightly missing due to covering more distance than 50m in one dt.

Kalman Filter Notes

Kalman Filter Notes

2 model system

1. Predictive model - position velocity, acceleration
2. Sensor data

Neither is perfect, kalman filter assesses which should be trusted more .

1. Start by initializing the filter. Give it a best guess for the state vector - relative position, velocity, and acceleration. Give it a guess for the P (covariance matrix). The diagonals represent how uncertain you are about each variable in the state vector.
2. Prediction.
 - a. $x_{pred} = F * x_{est}$; Propagate the estimate forward in time using the physics derived in the state transformation matrix. For this filter, I used constant-acceleration kinematics. This is the state prediction.
 - b. $P_{pred} = F * P_{est} * F' + Q$; this is the covariance prediction. It rotates the uncertainty through the state dynamics and adds the process noise. In this model, I have jerk modeled as the process noise.
3. Innovation
 - a. $innovation = rel_pos - H * x_{pred}$; this part finds the difference between the model prediction and the noisy sensor data. This assesses how well the model is approximating position. If innovation is small, then the model is doing well. If it's large, the model is not doing well, or the sensor is noisy. The filter has to figure out which one.
4. Kalman Gain

- a. $S = H * P_pred * H' + R$; this is the total innovation covariance. It normalizes gain by combining both sources of uncertainty.
 - b. $K = P_pred * H' / S$; This is the Kalman gain and is the most important part of the filter. It is basically (prediction uncertainty)/(total uncertainty). If the model is uncertain, and the sensor is better, then K will be large, and the sensor should be trusted more. If K is small, the sensor is noisy and the model should be trusted more.
5. Update
- a. $x_est = x_pred + K * innovation$; correct the state vector prediction by the innovation scaled by kalman gain.
 - b. $P_est = (eye(9) - K * H) * P_pred * (eye(9) - K * H)' + K * R * K'$; This is the Joseph form for covariance update, and it is more stable than the simpler form.

Summary:

The Kalman filter blends a physics-based prediction model with noisy measurement data and weights the data sets according to their relative uncertainty. It then updates the uncertainty estimates recursively, so the weights adjust over time.

X - state vector. Estimate of the system state, 9x1 representing relative p, v, a.

P - state covariance matrix. Uncertainty about the state vector.

F - State Transition Matrix. Propagates state estimation forward in time using the physics of the model. In my case, it is simple kinematics.

Q - Process Noise Covariance Matrix. Represents uncertainty in the model due to unknown inputs. In this case, it is the unknown target jerk.

R - Measurement Noise Covariance Matrix. Represents uncertainty in sensor readings. Diagonal standard deviation matrix.

H - Measurement/Observation Matrix. Maps full state vector to what the sensor actually observes.

Innovation - Innovation Vector (Measurement residual). The difference in what the sensor collected and what the model predicted.

S - Innovation Covariance Matrix. Total expected variance of the innovation. Combines predicted uncertainty and sensor uncertainty projected into the measurement space.

K - Kalman Gain Matrix. Weighs how much the innovation should correct the prediction. Balances trust in the model and sensor adaptively.

INTERCEPT: Within Range at 9.1m, $t = 76.6\text{s}$
Transitioned From Boost -->Glide at 48.0s

